



INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



REVISÃO DE ORIENTAÇÃO A OBJETO

Paradigma de POO Aplicada a Linguagem PHP



CSTSI - V SEMESTRE
DESENVOLVIMENTO DE APLICAÇÕES ORIENTADAS A OBJETOS
Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br
Março 2025

O que vamos (re)ver



- **Apresentação da disciplina**
 - Plano de Ensino
 - Cronograma
- **Configuração do ambiente e ferramentas**
 - Instalação do PHP 8.4
 - Instalação do Composer
 - Utilização do composer
- **Tópicos sobre POO aplicados a linguagem PHP:**
 - Namespace, Classes e Objetos;
 - Métodos, Construtor e Destrutor;
 - Modificadores de acesso, Getters e Setters;
 - Herança e Traits
 - Classe Abstrata e Interfaces
 - Polimorfismo
 - Exceptions e Try/Catch
- **Atividades práticas**



Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.



- Programação focada nos **DADOS** sua representação/manipulação na memória.
- A **CLASSE** define como estes dados serão representados e manipulados:
Exemplo:
 - Vamos desenvolver uma aplicação para calcular o *IMC* (*índice de massa corpórea*) de uma pessoa.
 - Uma **pessoa** possui diversas propriedades como nome, cor dos olhos, cor do cabelo, tipo de cabelo, idade, peso, altura, etc. No entanto, para a nossa aplicação, os dados relevantes são: **nome**, **idade**, **peso** e **altura**;
 - Para representarmos uma pessoa em nosso aplicativo podemos abstrair suas características na classe **Pessoa** com as propriedades: **\$nome**, **\$idade**, **\$peso** e **\$altura**;

Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.



- Implementação da classe **Pessoa**:

```
1  <?php
2
3  class Pessoa {
4      public $nome, $idade, $peso, $altura;
5  }
```

- Porém esta **classe** apenas descreve os dados que representam uma pessoa.
- Para utilizarmos esta **classe**, ou este modelo de “**pessoa**” em nosso programa, devemos **instanciar** esta classe em memória.
- Um **instância** da classe que representa uma pessoa é um **OBJETO** do tipo **Pessoa**.

Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.

- **OBJETO** é nada mais do que áreas reservadas de memória que possuem os dados de uma classe específica e referências para os métodos que manipulam estes dados.
- Ou seja, um **OBJETO** é a *instância* de uma determinada *classe* em memória.
- Através do objeto podemos acessar as suas propriedades, como dos dados da classe **Pessoa**.
- Podemos também instanciar mais de um objeto baseado em uma mesma classe.

```
1  <?php
2
3  class Pessoa {
4      public $nome,
5             $idade,
6             $peso,
7             $altura;
8  }
9
10 $obj = new Pessoa;
11 $obj->nome = "Joao";
12 $obj->idade= 34;
13
14 var_dump($obj);
```

Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.

- Veja o exemplo do **\$obj** na figura ao lado.
- Na **linha 10** criamos uma instância da **classe Pessoa** e atribuímos esta instância à variável **\$obj**;
- Nas linhas seguintes, acessamos seus atributos/propriedades e atribuímos valores;
- Na última linha usamos a função **var_dump** do **php** para visualizar os dados definidos anteriormente.

```
1  <?php
2
3  class Pessoa {
4      public $nome,
5             $idade,
6             $peso,
7             $altura;
8  }
9
10 $obj = new Pessoa;
11 $obj->nome = "Joao";
12 $obj->idade= 34;
13
14 var_dump($obj);
```

Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.



- Ao lado observamos a execução do nosso script de exemplo com a classe **Pessoa** e seu objeto **\$obj**;
- A saída da função **var_dump** nos mostra também o tipo de dado da variável passada como argumento (**\$obj**).
- Para **\$obj** podemos observar que se trata de um **objeto** (**object**) da classe **Pessoa**.
- Também podemos ver os **atributos** do objeto **\$obj** e seus respectivos tipos de dados.

```
<?php 8.1.7
$ php Pessoa.php
object(Pessoa)#1 (4) {
    ["nome"]=>
    string(4) "Joao"
    ["idade"]=>
    int(34)
    ["peso"]=>
    NULL
    ["altura"]=>
    NULL
}
```

Classes e Objetos

Revisão de conceitos básicos de POO aplicados ao PHP.



- Instanciando dois objetos da classe **Pessoa** :

```
1 <?php
2
3 class Pessoa {
4     public $nome, $idade, $peso, $altura;
5 }
6
7 $pessoaUm = new Pessoa;
8 $pessoaUm->nome = "Joao";
9 $pessoaUm->idade= 34;
10
11 $pessoaDois = new Pessoa;
12 $pessoaDois->nome = "Maria";
13 $pessoaDois->peso= 60;
14 $pessoaDois->altura= 1.6;
15
16 var_dump($pessoaUm, $pessoaDois);
```

```
_01_exemplo_imc <?php 8.1.7
$ php Pessoa.php
object(Pessoa)#1 (4) {
    ["nome"]=>
    string(4) "Joao"
    ["idade"]=>
    int(34)
    ["peso"]=>
    NULL
    ["altura"]=>
    NULL
}
object(Pessoa)#2 (4) {
    ["nome"]=>
    string(5) "Maria"
    ["idade"]=>
    NULL
    ["peso"]=>
    int(60)
    ["altura"]=>
    float(1.6)
}
```

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- O método construtor (`__construct`) recebe parâmetros para inicializar atributos de uma classe.
- Podemos criar um construtor para inicializar os atributos da classe **Pessoa**:

```
1  <?php
2
3  class Pessoa{
4      public $nome, $idade, $peso, $altura;
5
6      function __construct($nome, $idade, $peso, $altura)
7      {
8          $this->nome = $nome;
9          $this->idade = $idade;
10         $this->peso = $peso;
11         $this->altura = $altura;
12     }
13 }
```

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- O **\$this** é uma referência ao objeto instanciado em memória.
- Desta forma, podemos então atribuir ou modificar os atributos do objeto que foi instanciado.

```
1  <?php
2
3  class Pessoa{
4      public $nome, $idade, $peso, $altura;
5
6      function __construct($nome, $idade, $peso, $altura)
7      {
8          $this->nome = $nome;
9          $this->idade = $idade;
10         $this->peso = $peso;
11         $this->altura = $altura;
12     }
13 }
```

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- Agora que temos um construtor, passamos os parâmetros durante a criação da nova instância da classe **Pessoa**:
- Podemos então inicializar dois objetos em memória passando os argumentos que definirão o estado de cada objeto.
- Observe que somos obrigados a passar todos os parâmetros que o construtor espera receber, por isso o uso do **null** na linha 1 da figura abaixo.

```
1  $pessoaUm = new Pessoa("Joao",34,null,null);
2  $pessoaDois = new Pessoa("Maria",24,60,1.6);
3
4  var_dump($pessoaUm, $pessoaDois);
```

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- Podemos tornar os parâmetros **\$peso** e **\$altura** opcionais definindo valores na assinatura do construtor, como mostra a figura abaixo (linha 4):

```
1 class Pessoa{
2     public $nome, $idade, $peso, $altura;
3
4     function __construct($nome, $idade, $peso=null, $altura=null)
5     {
6         $this->nome = $nome;
7         $this->idade = $idade;
8         $this->peso = $peso;
9         $this->altura = $altura;
10    }
11 }
12
13 $pessoaUm = new Pessoa("Joao",34);
```

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- O **php** possui o mesmo conceito de destrutor (**`__destruct`**) de linguagens como **C++**, basicamente se trata de uma função especial chamada quando o objeto é removido da memória:

```
1    public function __destruct()  
2    {  
3        echo "\n$this→nome foi destruído!";  
4    }  
5 }
```

- Situações em que este método poderá ser chamado:
 - atribuição de ***null*** à referência do objeto;
 - encerramento do script.

Métodos Construtor e Destrutor

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo do destrutor:

```
1 <?php
2
3 class Pessoa{
4     public $nome, $idade, $peso, $altura;
5
6     public function __construct($nome, $idade, $peso=null, $altura=null)
7     {
8         $this->nome = $nome;
9         $this->idade = $idade;
10        $this->peso = $peso;
11        $this->altura = $altura;
12    }
13
14    public function __destruct()
15    {
16        echo "\n$this->nome foi destruído!";
17    }
18 }
19
20 $pessoaUm = new Pessoa("Joao",34);
21 $pessoaDois = new Pessoa("Maria",24,60,1.6);
22
23 var_dump($pessoaUm, $pessoaDois);
```

```
$ php Pessoa.php
object(Pessoa)#1 (4) {
    ["nome"]=>
    string(4) "Joao"
    ["idade"]=>
    int(34)
    ["peso"]=>
    NULL
    ["altura"]=>
    NULL
}
object(Pessoa)#2 (4) {
    ["nome"]=>
    string(5) "Maria"
    ["idade"]=>
    int(24)
    ["peso"]=>
    int(60)
    ["altura"]=>
    float(1.6)
}

Maria foi destruído!
Joao foi destruído!%
```

Modificadores de acesso

Revisão de conceitos básicos de POO aplicados ao PHP.



- O PHP suporta os modificadores de acesso ***public***, ***protected*** e ***private***.
- ***Public***: é o padrão, quando nenhum modificador de acesso é definido, tanto para ***propriedades*** quanto para ***métodos***. Propriedades e métodos com esta definição são acessíveis fora do escopo da classe, bastando apenas a referência ao objeto e o sinal de seta (→);
- ***Protected***: define propriedades e método ***protegidos***, ou seja, são acessados somente por classes filhas, que herdam estas propriedades de suas classes superiores;
- ***Private***: propriedades e métodos acessados somente no escopo da classe, não são acessíveis por outros objetos ou programas fora da classe, assim como não são visíveis por outras classes na relação de herança.

Modificadores de acesso

Revisão de conceitos básicos de POO aplicados ao PHP.



- Vamos criar um atributo **privado** na nossa Classe **Pessoa** para armazenar o **IMC** (Índice de Massa Corpórea).
- Crie um método para calcular o valor do IMC, sendo o índice calculado conforme a fórmula: $imc = peso / altura^2$;

```
1 class Pessoa{  
2     public $nome, $idade, $peso, $altura;  
3     private $imc;  
4 }
```

- O método deverá ser público, para que possamos ter acesso ao cálculo.
- O imc poderá ser retornado ou impresso.

Modificadores de acesso

Revisão de conceitos básicos de POO aplicados ao PHP.



- Note que o primeiro objeto (***pessoaUm***) foi construído sem os dados de peso e altura, portanto o método deverá prever esta situação.

```
1  $pessoaUm = new Pessoa("Joao",34);
2  $pessoaDois = new Pessoa("Maria",24,60,1.6);
3
4  $pessoaUm->calcImc();
5  $pessoaDois->calcImc();
6  $pessoaDois->imc;
```

Modificadores de acesso

Revisão de conceitos básicos de POO aplicados ao PHP.



- Note que o primeiro objeto (***pessoaUm***) foi construído sem os dados de peso e altura, portanto o método deverá prever esta situação.

```
x echoes@echoes-Inspiron-15-3567 ...tsi_daoo/aula_01/cstsi_daoo_aula_01_exemplo_imc <?php 8.1.7
$ php Pessoa.php

O IMC Joao: Erro, informe o peso e a altura primeiro!

O IMC Maria: 23.4375
PHP Fatal error:  Uncaught Error: Cannot access private property Pessoa::$imc in /media/Dados/Projetos/tsi_daoo/aula_01/cstsi_daoo_aula_01_exemplo_imc/Pessoa.php:37
Stack trace:
#0 {main}
   thrown in /media/Dados/Projetos/tsi_daoo/aula_01/cstsi_daoo_aula_01_exemplo_imc/Pessoa.php on line 37

Maria foi destruído!
Joao foi destruído!%
```

- Note também o erro **na linha 37**, ao tentarmos acessar uma propriedade privada.

Getters e Setters

Revisão de conceitos básicos de POO aplicados ao PHP.



- **GETTER:**

São os métodos que provem acesso aos valores de propriedades inacessíveis, retornando estes valores.

```
1 public function getAltura(){
2     return $this->altura;
3 }
4
5 public function getPeso(){
6     return $this->peso;
7 }
```

Podem retornar os valores com algum tipo de formatação de saída por exemplo.

Getters e Setters

Revisão de conceitos básicos de POO aplicados ao PHP.



SETTER:

São os métodos que viabilizam a manipulação de propriedades inacessíveis.

Recebem como parâmetros os novos valores a serem atribuídos às propriedades privadas ou protegidas.

```
1 public function setAltura($altura)
2 {
3     $this->altura = $altura;
4 }
5
6 public function setPeso($peso)
7 {
8     $this->peso = $peso;
9 }
```

Getters e Setters

Revisão de conceitos básicos de POO aplicados ao PHP.



- Podemos modificar o exemplo definindo os valores de peso e altura para o primeiro objeto (*\$pessoaUm*).

```
1  $pessoaUm = new Pessoa("Joao", 34);  
2  $pessoaDois = new Pessoa("Maria", 24, 60, 1.6);  
3  
4  $pessoaUm->setAltura(1.7);  
5  $pessoaUm->setPeso(75);  
6  $pessoaUm->calcImc();  
7  $pessoaDois->calcImc();
```

Getters e Setters

Revisão de conceitos básicos de POO aplicados ao PHP.



- Não faz sentido o uso de método GETTERS e SETTERS em propriedades públicas.
- O **PHP** possui os métodos *mágicos* **set** e **get** para acesso á propriedades inacessíveis;
- **__get**: recebe um parâmetro para o nome da propriedade a ser retornada e retorna o valor desta, utilizando o **\$this** para o acesso.

```
1 public function __get($name){  
2     return $this→$name;  
3 }
```

- Implementando este método podemos acessar a propriedade **\$imc** que é **privada** em nossa classe **Pessoa**;

Getters e Setters

Revisão de conceitos básicos de POO aplicados ao PHP.



- **__set**: recebe um parâmetro para o nome da propriedade a ser modificado (**\$name**) e o valor (**\$value**) a ser atribuído;

```
1 public function __set($name,$value){  
2     $this->$name = $value;  
3 }
```

- Implementando este método podemos modificar a propriedade **\$imc** que é **privada** em nossa classe **Pessoa** simplesmente atribuindo um valor à propriedade;
 - **\$obj**→**propriedadePrivada** = **novoValor**; (inviável sem o **__set**)

Herança

Revisão de conceitos básicos de POO aplicados ao PHP.



- A **Herança** é um dos principais mecanismos do paradigma de programação Orientada a Objetos.
- Através de herança é possível **reutilizar** propriedades e comportamentos (métodos) entre objetos.
- Também provem uma melhor organização da aplicação com a **separação de responsabilidades** entre as Classes.
- Em PHP a herança é implementada via a palavra reservada **extends**.

```
1 Class Atleta extends Pessoa {  
2     public function __construct($nome,$altura, $peso)  
3     {  
4         parent::__construct($nome,$peso,$altura);  
5         $this->calcImc();  
6     }  
7 }
```



- Toda a referência a classe Pai se faz pela palavra reservada **parent**;
- O construtor da classe Paciente usa o **parent** para repassar os parâmetros à sua classe Pai (Pessoa);
- Neste caso poderemos acessar através de um objeto do tipo Paciente, todos as propriedades e métodos da classe Pessoa.
- Exemplo:

```
1 Class Atleta extends Pessoa {  
2     public function __construct($nome,$altura, $peso)  
3     {  
4         parent::__construct($nome,$peso,$altura);  
5         $this->calcImc();  
6     }  
7 }
```

Herança

Revisão de conceitos básicos de POO aplicados ao PHP.



- Exemplo:

```
1 <?php
2 include "classes/Pessoa.php";
3 include "classes/Atleta.php";
4
5 $jogador = new Atleta("Walter Kannemann",1.83,80);
6
7 $jogador->calcImc();
8 $jogador->showIMC();
```

```
os_poo/aula_01_exemplo_imc > <?php 8.1.7
```

```
$ php exemplo-02-heranca.php
```

```
IMC Walter Kannemann: 23.88844098062
```

```
Walter Kannemann foi destruído!
```



- Exemplo:

```
1  <?php
2  include "Pessoa.php";
3  include "Paciente.php";
4
5  $pa1 = new Paciente("Luizinho",1.8,80);
6
7  $pa1->calcImc();
8  $pa1->showIMC();
```

```
<?php 8.1.7
$ php exemplo-02-heranca.php
```

```
IMC Luizinho: 24.691358024691
Luizinho foi destruído!
```

Atividade de Revisão

Revisão de conceitos básicos de POO aplicados ao PHP.



- Implemente a classificação recomendada ao lado através de uma classe chamada **IMC**;
- A classe **IMC** deverá possuir os seguintes métodos estáticos:
 - **calc**: recebe um objeto do tipo Paciente e retorna o valor de seu imc.
 - **classifica**: recebe um objeto do tipo Paciente e retornar sua classificação como **string** de acordo com as faixas de imc da imagem ao lado.



```
class App{  
    public static function init() :void {
```

```
App::init();
```



- LINKS SOBRE PDO:
 - <https://www.php.net/manual/en/book.pdo.php>
 - <https://www.php.net/manual/en/pdo.drivers.php>
 - <https://www.php.net/manual/en/ref.pdo-mysql.connection.php>
 - <https://www.php.net/manual/en/ref.pdo-pgsql.connection.php>
 - https://www.php.net/manual/pt_BR/language.oop5.interfaces.php
 - https://www.php.net/manual/pt_BR/language.oop5.decon.php
- DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009